

MatchProof product matching proposal

UAVS Hackathon 2026 | Round 1 | FPT Australasia

1. Executive summary

We propose MatchProof, a merchant-side API that interprets requests from AI shopping agents and returns relevant products with evidence for every mandatory condition. It addresses a specific failure in product discovery: a recommendation may sound suitable while exceeding a budget, containing an excluded material, or relying on an attribute the catalog never confirms.

Our prototype will combine AI intent interpretation, explicit condition checks and semantic ranking. Each condition receives one of three outcomes: satisfied, violated or unknown. Only products with evidence satisfying every mandatory condition enter the confirmed recommendation list. We will test the approach against keyword and semantic-search baselines, measuring invalid recommendations alongside useful matches.

Team information

Team name: Mát nét Team leader: Huy Duc Vu

Member	Full name	University	Contact email
1 Leader	Huy Duc Vu	UTS	vuhuyduc03@gmail.com
2	Nam Khanh Tran	UNSW	
3	Anh Quan Tran	UOW	
4	Bao Tong	QUT	

Team leader phone: +61 410 899 204

Chosen challenge and reason

We select FPT Australasia's The B2A Shift: Adapting Retail for AI Shopping Agents. Its emphasis on understanding multi-condition requests makes discovery and matching a focused, testable stage of the purchasing journey. FPT's digital commerce and AI capabilities provide a plausible integration context for a reusable merchant service.

The primary users are retailer commerce engineers and catalog managers; the API consumer is an external shopping or procurement agent. The first demonstration uses a team-authored headphone catalog. The system's scope is matching logic, with category-specific attributes supplied through a schema. Checkout, payments and building the buyer's agent are outside the MVP.

2. Problem framing and supporting research

A request such as “wireless over-ear headphones under A\$150, at least 30 hours of battery life, no animal leather, preferably foldable” combines category, connectivity, numerical limits, negation and preference. A product that meets five conditions can still be unsuitable because it fails the sixth. The merchant needs to expose that distinction in a form an agent can inspect.

What the research establishes

Negation requires more than keyword overlap. Researchers identify a vocabulary gap between excluded features in queries and the language used in product descriptions. Their product-search research supports treating exclusions as an explicit interpretation problem. It does not establish the failure rate of FPT's clients or the effectiveness of MatchProof.

Semantic retrieval is an established technique. Walmart researchers describe a deployed hybrid of lexical and embedding-based retrieval for complex product searches. This supports a practical retrieval baseline. MatchProof will evaluate whether a separate evidence check improves condition compliance while retaining useful results.

Merchant infrastructure is also advancing. Shopify Catalog structures and distributes product attributes to AI channels. UCP defines catalog search with text queries and filters. These services provide useful integration patterns. Their published interfaces do not by themselves establish that a given implementation verifies every natural-language condition; we will not claim existing platforms lack such capabilities.

FPT Australasia lists AI, data and analytics, and digital commerce among its services. Our inference is that a matching component with inspectable decisions could fit retailer integration work. Access to client data, a commercial deployment and interest from a particular retailer remain assumptions to verify.

Proposed contribution and technical alternatives

Approach	Role in this proposal
Keyword search	Cheap, transparent baseline; test vocabulary mismatch and exclusions.
Semantic search	Relevance baseline; similarity alone is not our evidence of eligibility.
LLM with basic filters	Strong comparison using the same parser; isolates the benefit of evidence handling.
MatchProof	Adds three-outcome checks, evidence references and explicit separation of eligibility from ranking.

Our contribution is the combined behavior and its evaluation, rather than a new foundation model or a claim to have invented semantic search. We will publish the supported operators, decision traces and failure cases so judges can inspect how a request becomes a recommendation.

3. Matching logic and first MVP

The first feature is POST /match over a fixed catalog. It accepts an English request and an explicit currency, then returns up to three confirmed products, the interpreted requirements and concise diagnostics. Supported operations are equality, exclusions, minimum and maximum values, and AND combinations of mandatory conditions. Explicit preferences affect ranking. Unsupported or contradictory requests receive a needs_clarification response.

Interpret the request

A schema-constrained LLM maps phrases into attribute, operator, value, unit and importance, retaining the source phrase for each condition. “Under A\$150” is strictly less than 15,000 cents; “at least 30 hours” is inclusive. “Preferably foldable” stays optional. Unqualified requested features are mandatory under the documented MVP policy. Ambiguous scope or an unmapped phrase is returned for clarification rather than omitted.

Check evidence before ranking

The evaluator examines every variant in the small catalog. A condition is satisfied only when an approved fact supports it; contradictory facts or missing information produce unknown. An explicit opposing fact produces violated. Absence of “leather” from a description is insufficient to establish “no animal leather.” That condition needs complete material coverage for relevant product parts. All mandatory conditions must be satisfied for eligibility.

Eligible products are ordered first by the number of explicit preferences satisfied, then by semantic similarity between the request's use case and the product description, with product ID as a stable tie-breaker. Similarity never overrides a failed condition. Semantic context such as “for commuting” influences relevance without inventing a mandatory feature.

Illustrative decision on a sample catalog

All three fictional products below are wireless and over-ear. Prices are in AUD and battery hours use the same stated playback mode.

Variant	Price and battery	Material evidence	Decision
MP101	A\$139 40 hours	Animal leather present	Violated
MP102	A\$129 35 hours	Composition incomplete	Unknown
MP103	A\$145 32 hours	Complete record No animal leather	Satisfied

For the example request, MP103 is eligible. MP101 and MP102 remain visible as excluded diagnostics, not recommended substitutes. If nothing qualifies, the API reports no_verified_match. “Proof” means a traceable check against the catalog snapshot; it is not independent certification of manufacturer claims.

4. Architecture and API design

The merchant owns catalog preparation and the matching service. A small test client represents the requesting agent. A single service keeps the prototype easy to run and inspect; catalog ingestion occurs before the timed demonstration.

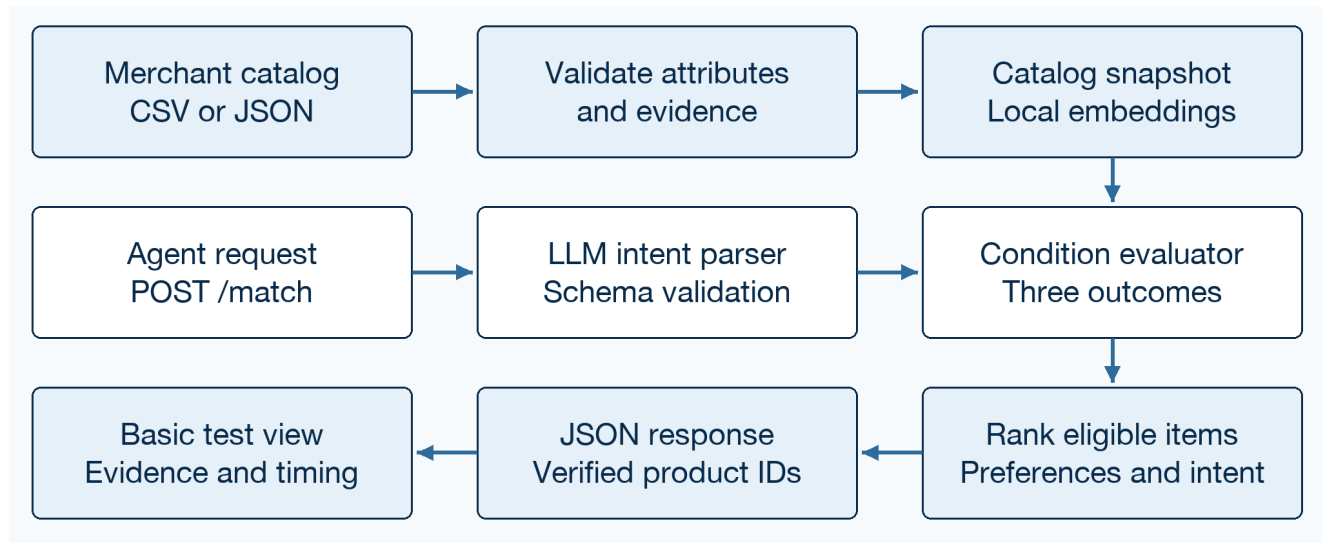


Figure 1. Merchant catalog preparation and request processing.

Components and data ownership

FastAPI with Pydantic provides typed request and response validation. SQLite stores variant facts and provenance. A hosted Gemini Flash-Lite model interprets requests; local all-MiniLM-L6-v2 embeddings provide semantic similarity. The MVP scans all catalog variants before ranking, avoiding a vector-search cutoff that could hide a valid item. A single pinned snapshot governs each response.

Request and response contract

Inputs are request_id, catalog_id, query and currency; the catalog is restricted to the authenticated merchant. Each parsed condition includes a source phrase. Each result contains product_id, variant_id, rank and evidence records with attribute, observed value, condition outcome, source reference and catalog version. Top-level status is matched, no_verified_match, needs_clarification or service_error.

Catalog records include integer prices, currency, availability, normalized attributes and units, source_id, source location, verification status and timestamps. Evidence belongs to the exact variant. Missing and conflicting facts stay unknown. A product's inclusion in stock is checked against the snapshot, with final availability and price revalidation left to checkout.

Failure behavior and integration

Validated parser output enters the evaluator. One bounded retry is allowed; parser or catalog failure returns service_error. Catalog text cannot trigger tools. API keys remain server-side; imports require authorization and logs omit personal data. A later commerce adapter can expose this service through existing protocols.

5. Data plan and evaluation

We will author 60 fictional catalog variants and a labeled set of 80 requests. Product records will deliberately include synonyms, unit differences, exclusions, unknown facts and conflicts. This follows the supplied clarification permitting sample catalogs. No customer histories or private retailer data are needed. Synthetic records provide controlled ground truth, but cannot establish commercial impact or generalization to real catalogs.

Split the requests into 30 development and 50 held-out cases. Separate paraphrase families across the split and freeze rules before the final run. Held-out coverage will include multi-condition matches, negations, strict numerical boundaries, preferences, missing evidence, contradictions, ambiguous requests and cases with no valid product. Two members will independently review condition labels and acceptable product sets, resolving disagreements before scoring.

Run keyword, embedding-only, LLM plus basic filters, and MatchProof on the same inputs and catalog. The basic-filter baseline shares the parser and ranking model but omits the full evidence checks. Add an evaluator-only run using human-labeled intent to distinguish interpretation errors from checking errors. Report all results, including abstentions and unsuccessful cases.

Measure	Proposed acceptance target
Intent accuracy	At least 90% of held-out requests have every condition parsed correctly.
Invalid confirmed results	Zero observed violations of human-labeled mandatory conditions; show counts and denominator.
Useful match rate	At least 90% of answerable held-out requests return a valid product in the top three.
Uncertainty and evidence	All designed unknown/conflict cases handled correctly; all confirmations cite valid facts.
Latency and cost	95th-percentile response within 3 seconds over 100 warm requests; mean model cost below US\$0.005.

These are targets, not measured results. An empty answer cannot improve useful-match rate; if no products are returned at all, the violation rate is reported as undefined. Report performance by case type and retain exact traces. Zero observed errors on a small test does not establish a universal guarantee.

Thirty second demonstration

0–5 seconds: submit the multi-condition request. 5–12: reveal the parsed conditions and mandatory/preferred distinction. 12–22: inspect MP101's violation, MP102's unknown material evidence and MP103's supported match. 22–30: lower the cap to A\$140 and show no_verified_match. Display live timing; any contingency replay must be clearly labeled.

6. Feasibility, costs and expected value

The sample database, evaluator, interface and local embeddings will run on a laptop without a dedicated GPU or model training. all-MiniLM-L6-v2 is Apache-2.0 licensed. Pin versions and retain attribution. Real merchant catalogs require authorized access, reviewed evidence and an agreed refresh policy.

Preliminary operating budget

For planning, Gemini 3.5 Flash-Lite standard text pricing is US\$0.30 per million input tokens and US\$2.50 per million output tokens. Assume one parsing call using 1,800 input and 600 total output tokens, including any thinking tokens. This gives US\$0.00204 per request before retries. No web grounding or hosted embedding fees are assumed.

Item	Basis	Estimate in USD
Model calls	10,000 requests per month	20.40
Retry allowance	10% additional model usage	2.04
Compute and storage	Planning allowance, not a vendor quote	20.00
Contingency	Additional monthly reserve	10.00
Monthly total	Pilot assumption before tax and labor	52.44

Set a US\$60 monthly planning ceiling for that workload. A local prototype with 2,000 calls would cost about US\$4.49 in model usage including the retry allowance; reserve US\$10 for experimentation. Actual tokens, hosting capacity and prices must be measured before deployment.

Merchant value to validate

We will test whether retailers receive fewer unsuitable recommendations and identify missing catalog facts more quickly. A pilot would measure successful matches and manual review time. Illustratively, 1,000 requests with 20% requiring a 60-second check consume 200 minutes; halving those checks saves 100 minutes. These assumptions require validation with an authorized retailer.

Main risks and mitigations

Parser omissions can make an incorrect request appear valid. Retain source phrases and test negation, completeness and numerical boundaries. Track useful matches alongside abstentions so missing catalog data does not hide poor coverage. Merchant approval and freshness remain necessary because correct logic cannot establish the truth of source data.

Larger catalogs will need indexed filters and incremental imports, with retrieval recall tested before approximate search. New categories require attribute schemas and separate validation.

7. Roadmap, assumptions and open questions

Our preliminary build estimate is 16 working hours for four members, with 64 person-hours available. The schedule is an implementation estimate, subject to the full brief. The critical path is a validated catalog, a correct evaluator, then integration of language interpretation and ranking.

Window	Work and proposed ownership	Completion evidence
0–4 h	Member 1: schema and request labels. Member 2: sample data and evaluator. Member 3: parser. Member 4: API and test view.	Agreed schema and a working vertical slice.
4–8 h	Integrate parsing, evidence checks, semantic ranking and response records.	One live request returns traceable results.
8–12 h	Complete data review, baseline comparisons and development-case fixes.	Frozen rules and labeled held-out cases.
12–16 h	Run final evaluation, inspect failures, document setup and rehearse demo.	Reproducible report and a 30-second demonstration.

If time is constrained, preserve interpretation, evidence checking and the comparison experiment; reduce interface polish. A second category and commerce-platform adapters are later work. A post-hackathon pilot would first validate an authorized merchant catalog, then run alongside existing search before any customer-facing rollout.

Assumptions to validate

The first catalog is a single merchant's English-language dataset priced in AUD. Sample prices are item prices; a request for delivered cost requires data the MVP does not hold. Merchant records can carry variant-level evidence, material completeness and update timestamps. Agents can consume JSON and accept explicit uncertainty. These are design assumptions, not confirmed properties of FPT client systems.

Questions for the Problem Setter

1. Which initial retailer or agent workflow would best demonstrate value, and which multi-condition requests cause the most difficulty?
2. What product schema, evidence sources, variant detail and freshness should advancing teams expect, and how should conflicting facts be resolved?
3. What balance between abstention and coverage is acceptable, and should mandatory delivered price, inventory or policy conditions be included?
4. What query volume, response time, deployment environment and benchmark would make a prototype credible for FPT's intended use?

We will use the organizer's talkshow notes and available clarification to refine these assumptions. The proposed outcome is a small, inspectable matching service and honest evaluation evidence showing when it can support a recommendation and when it must withhold one.